

拓扑 AC | CSP-S 线上联测

TopoAC Algorithm Contest

第二轮 第一场

题目名称	不交叉配对	二进制生长	异或替换	沿途密钥
题目类型	传统型	传统型	传统型	传统型
目录	pair	growth	xorreplace	keytrail
可执行文件名	pair	growth	xorreplace	keytrail
输入文件名	pair.in	growth.in	xorreplace.in	keytrail.in
输出文件名	pair.out	growth.out	xorreplace.out	keytrail.out
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	30	34	25	25
子任务数目	3	6	3	7
测试点是否等分	否	否	否	是

提交源程序文件名

对于 C++ 语言	pair.cpp	growth.cpp	xorreplace.cpp	keytrail.cpp
-----------	----------	------------	----------------	--------------

编译选项

对于 C++ 语言	-O2 -std=c++14 -Wall
-----------	----------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 `0`。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 禁止在源代码中改变编译器参数（如使用 `#pragma` 命令），禁止使用系统结构相关指令（如内联汇编）和其他可能造成不公平的方法。
7. 请注意所有题目均使用子任务计分，只有通过属于该子任务的全部数据后，才能获得该子任务的分数。

不交叉配对 (pair)

【题目描述】

活动桌上放着两排标记卡。第一排有 n 张，第二排有 m 张，每张卡片上写着一个大写英文字母。

你可以把两排中字符相同的卡片连成一对。每张卡片至多使用一次，并且连线不能交叉。形式化地，若选出的配对依次为

$$(i_1, j_1), (i_2, j_2), \dots, (i_k, j_k),$$

则必须满足

$$1 \leq i_1 < i_2 < \dots < i_k \leq n, \quad 1 \leq j_1 < j_2 < \dots < j_k \leq m,$$

并且每个 t 都有 $S_1[i_t] = S_2[j_t]$ 。

请你求最多能选出多少对卡片。若没有任何字符能够配对，答案为 0。

【输入格式】

从 `pair.in` 文件读入数据。

第一行包含两个整数 n, m 。

第二行包含一个长度为 n 、仅由 A 到 Z 组成的字符串 S_1 。

第三行包含一个长度为 m 、仅由 A 到 Z 组成的字符串 S_2 。

【输出格式】

输出到 `pair.out` 文件。

输出一个整数，表示最多能够选出的配对数。

【样例 1 输入】

```
1 2 5
2 AB
3 BAABB
```

【样例 1 输出】

```
1 2
```

【样例 1 解释】

可以依次配对 S_1 的 A、B 与 S_2 的第 2、第 4 个字符。

【样例 2 输入】

```
1 3 5
2 ABA
3 AABBA
```

【样例 2 输出】

```
1 3
```

【样例 2 解释】

重复字母可能有多种选择；只要两侧下标均严格递增即可。

【样例 3 输入】

```
1 3 3
2 ABC
3 DEF
```

【样例 3 输出】

```
1 0
```

【数据范围】

对于所有数据：

- $1 \leq n \leq 1000$ ；
- $1 \leq m \leq 10^6$ ；
- S_1, S_2 仅含大写英文字母。

本题采用子任务捆绑计分。每个测试点恰好属于下表中的一个子任务；一个子任务内所有测试点均通过才能获得该子任务分数。

子任务编号	分值	附加约束
1	30	$m \leq 1000$
2	20	$m > 1000$ ，且 S_1 中所有字符相同
3	50	$m > 1000$ ，且 S_1 至少包含两种字符

二进制生长 (growth)

【题目描述】

实验室中有 n 个正整数编号 $a_1, a_2, \dots, a_n$ 。一次生长操作可以任选一个编号 x ，把它变成 $2x$ 或 $2x + 1$ 。每个编号都可以执行任意多次操作，也可以不操作。

完成全部操作后，记所有编号中的最大值为 M ，最小值为 m 。请你求 $M - m$ 的最小可能值。

【输入格式】

从 *growth.in* 文件读入数据。

第一行包含一个整数 n 。

第二行包含 n 个正整数 $a_1, a_2, \dots, a_n$ 。

【输出格式】

输出到 *growth.out* 文件。

输出一个非负整数，表示最终极差的最小可能值。

【样例 1 输入】

```
1 5
2 1 5 20 22 29
```

【样例 1 输出】

```
1 9
```

【样例 1 解释】

一种达到该答案的方式是把 1 依次变成 3, 6, 12, 24，把 5 依次变成 10, 21。最终编号可以为 20, 21, 22, 24, 29，极差为 9。

【样例 2 输入】

```
1 10
2 1 2 3 4 5 6 7 8 9 10
```

【样例 2 输出】

```
1 4
```

【样例 3 输入】

```
1 10
2 10 11 12 13 14 18 19 20 22 24
```

【样例 3 输出】

```
1 10
```

【数据范围与子任务】

- 对于所有数据：
- $2 \leq n \leq 200000$ ；
 - $1 \leq a_i \leq 10^{18}$ 。
- 本题采用子任务捆绑计分。

子任务编号	分值	附加约束
1	10	$n \leq 10$ 且 $a_i \leq 100$
2	15	$n \leq 500$ 且 $a_i \leq 500$
3	20	$a_i \leq 10^5$
4	20	$n \leq 1000$
5	32	无附加约束
6	3	$n \leq 6$

异或替换 (xorreplace)

【题目描述】

熊熊维护着一个长度为 n 的非负整数序列：

$$a_1, a_2, \dots, a_n.$$

一次操作需要先选择一个下标 u ，其中 $1 \leq u \leq n$ 。设操作开始时整个序列的按位异或和为

$$w = a_1 \oplus a_2 \oplus \dots \oplus a_n,$$

然后把 a_u 修改为 w 。同一个下标可以被选择多次。

现在给定初始序列 a 和目标序列 b 。数据保证可以通过有限次操作把 a 变为 b ，请你求所需操作次数的最小值。

【输入格式】

从 `xorreplace.in` 文件读入数据。

第一行包含一个正整数 n 。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ 。

第三行包含 n 个整数 $b_1, b_2, \dots, b_n$ 。

【输出格式】

输出到 `xorreplace.out` 文件。

输出一个整数，表示把 a 变为 b 所需的最少操作次数。

【样例 1 输入】

```
1 5
2 3 3 5 6 9
3 10 3 5 6 9
```

【样例 1 输出】

```
1 1
```

【样例 1 解释】

初始序列的按位异或和为 10。选择 $u = 1$ 执行一次操作后，序列即变为目标序列。

【样例 2 输入】

```
1 4
2 1 2 4 7
3 2 1 7 4
```

【样例 2 输出】

```
1 6
```

【样例 3】

输入与输出分别见 sample 目录下的 `xorreplace/xorreplace3.in` 和 `xorreplace/xorreplace3.ans`。

【数据范围】

- 对于所有数据：
- $1 \leq n \leq 2 \times 10^5$ ；
 - $0 \leq a_i, b_i < 2^{30}$ ；
 - 保证存在一种有限步操作方案可以把 a 变为 b 。
- 本题采用捆绑测试。只有通过一个子任务的全部测试，才能获得该子任务的分数。

子任务编号	分值	附加约束
1	30	$n \leq 10$
2	30	$n > 10$ ，存在整数 $k \geq 2$ 使 $n = 2^k - 1$ ，并且对所有 $1 \leq i \leq n$ 都有 $a_i = i$
3	40	$n > 10$ ，并且至少满足一项：不存在整数 $k \geq 2$ 使 $n = 2^k - 1$ ； 存在 $1 \leq i \leq n$ 使 $a_i \neq i$

沿途密钥 (keytrail)

【题目描述】

一片区域内有 n 个站点，由 $n - 1$ 条双向道路连接。任意两个站点之间都能互相到达，因此这些站点和道路构成一棵树。

站点中存放着 m 份档案。第 i 份档案位于站点 y_i ，打开它的密钥位于站点 x_i ，这份档案的价值为 w_i 。价值可能为负数。

你需要选择一个起点和一个终点，沿两点间的简单路径完成一次行程。起点和终点可以相同。行程经过每个站点时，按以下顺序执行操作：

1. 收集该站点存放的所有密钥；
2. 处理该站点中所有已经取得对应密钥的档案，并将这些档案的价值计入总价值。

起点也视为行程经过的站点，并执行上述操作。每份档案只在到达其所在站点时检查一次：如果此时还没有对应密钥，本次行程不会再处理这份档案。在第 2 步中已经取得对应密钥的档案必须全部处理，不能因为其价值为负而跳过。

请你求出一次行程能够获得的最大总价值。

【输入格式】

从 `keytrail.in` 文件读入数据。

第一行包含两个正整数 n, m ，分别表示站点数和档案数。

接下来 $n - 1$ 行，每行包含两个整数 a, b ，表示站点 a 与站点 b 之间有一条双向道路。

接下来 m 行，第 i 行包含三个整数 x_i, y_i, w_i ，依次表示第 i 份档案的密钥位置、档案位置和价值。

【输出格式】

输出到 `keytrail.out` 文件。

输出一个整数，表示能够获得的最大总价值。

【样例 1 输入】

```
1 3 2
2 1 2
3 2 3
4 1 3 8
5 3 1 5
```

【样例 1 输出】

```
1 8
```

【样例 1 解释】

选择从站点 1 到站点 3 的行程，可以先取得第 1 份档案的密钥，再在站点 3 处理这份档案，得到价值 8。如果沿相反方向行进，则只能处理价值为 5 的第 2 份档案。

【样例 2 输入】

```
1 5 5
2 1 2
3 2 3
4 2 4
5 4 5
6 1 3 7
7 2 2 -3
8 4 5 6
9 5 1 10
10 3 5 9
```

【样例 2 输出】

```
1 12
```

【样例 2 解释】

选择行程 $3 \rightarrow 2 \rightarrow 4 \rightarrow 5$ ，会依次计入价值 -3 、 6 和 9 ，总价值为 12 。

【样例 3】

输入与输出分别见 sample 目录下的 *keytrail/keytrail3.in* 和 *keytrail/keytrail3.ans*。

【数据范围】

对于所有数据，保证：

- $1 \leq n, m \leq 10^5$ ；
- $1 \leq a, b, x_i, y_i \leq n$ ；
- $-1000 \leq w_i \leq 1000$ ；
- 输入的 $n - 1$ 条道路构成一棵树；
- 每份档案相互独立；即使多份档案的 x_i, y_i, w_i 完全相同，也要分别处理并计入价值；
- 最优答案大于 0。

本题采用捆绑测试。只有通过一个子任务的全部测试，才能获得该子任务的分数。

子任务编号	分值	附加约束
1	8	$n, m \leq 10$
2	16	$n, m \leq 1000$
3	12	$m \leq 15$
4	20	对所有档案均有 $x_i = y_i$
5	12	对所有档案均有 $x_i = 1$
6	16	每个站点至多与两个站点直接相连
7	16	无附加约束